

Darbar Sweets ERP — Project Blueprint

Release: 1.6.6 · **Date:** 7 September 2026 · **Location:** Sukkur, Sindh

This blueprint brings together the PRD, TRD, application flow, UI/UX rules, backend schema, and completed implementation phases. Version 1.6.6 follows the confirmed release instruction. Security hardening preserves the existing UI and business calculations.

Implementation note: The current Prisma schema and packaged desktop use SQLite. PostgreSQL 15/Docker remains an optional legacy deployment/backup integration, rather than a desktop prerequisite. The requested architecture reference is retained below with that distinction. No database migration was made for this release.

1 — PRD: Product Requirement Document

One-line pitch

Darbar Sweets ERP helps the Darbar Sweets halwai shop in Sukkur, Sindh manage POS billing, inventory, production, HR, and accounting from one desktop application.

The problem

Small sweet shops manage sales on paper, track stock manually, and have no visibility into production costs, employee wages, or profit margins. Errors are common and there is no audit trail.

Target user

- **Owner/Admin:** Full access, including reports, accounting, HR, payroll, and settings.
- **Production Manager:** Inventory, recipes, production orders, supplier information, and production dashboard figures.
- **Cashier:** POS billing, orders, customers, and sales returns.

The existing Staff role is retained; it has no assigned application tabs. Sensitive employee, salary, and accounting data is restricted by the backend, not only by navigation visibility.

Core features — v1, built

1. POS with receipt printing, token system, and delivery charges.
2. Inventory with batch/expiry, stock in/out, and cost tracking.
3. Recipe/BOM with weighted-average production costing.
4. HR: employees, attendance, leave, and daily/monthly payroll.
5. Double-entry accounting with automatic journal entries for supported business transactions.
6. Advance orders, customer profiles, and supplier ledger.
7. Backup/restore, automatic update support, and role-based access.

Out of scope

- Online ordering and ecommerce.
- Multi-branch support.
- Biometric attendance.
- WhatsApp/SMS notifications.
- Loyalty program.

Success criteria

- Admin can generate a daily P&L report in under 30 seconds.
- Cashier can complete a POS sale in under 60 seconds.
- Production Manager can complete a production order and see cost per kg.

These are product acceptance targets, not measured performance claims from this security audit.

2 — TRD: Technical Requirement Document

Stack — preserved

Layer	Technology
Frontend	React 18, TypeScript, Vite, Tailwind CSS, React Query, Zustand
Backend	Node.js, Express 4, TypeScript
Current desktop database	SQLite with Prisma ORM
Requested architecture reference	PostgreSQL 15 with Prisma; optional legacy Docker support exists
Authentication	JWT: 15-minute access, 30-day refresh; bcryptjs
Desktop	Electron, electron-builder, electron-updater
Hosting	Local host machine; frontend served by the bundled Express server

Third-party packages

Package	Purpose
Prisma	ORM, schema generation, and database operations
bcryptjs	Password hashing, cost factor 12
jsonwebtoken	JWT signing and signature verification
helmet	CSP and HTTP security headers
express-rate-limit	Auth and API rate limiting
express-validator	Auth and user-management request validation
express-async-errors	Forward asynchronous Express 4 route errors to the global handler
multer	Image and backup uploads
node-cron	Backup scheduler
dayjs	Date formatting and calculations
jsPDF / html2canvas	Receipt and document rendering

Constraints

- Core shop operations work offline, with optional local-network access.
- Update checks/downloads require a network connection; they are separate from offline billing.
- Docker Desktop is needed only for the optional Docker/PostgreSQL workflow. Normal desktop use needs no Docker setup.
- Single shop, single branch.
- Windows 10/11 desktop distribution.
- Do not introduce GraphQL, Redis, microservices, or Firebase.

Production security configuration

- Critical environment variables: DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET. Missing or weak JWT configuration prevents startup.
- Local secrets live in ignored `server/.env`; the packaged desktop generates independent secrets in its `userData/runtime/server/.env`.
- Initial passwords come from SEED_ADMIN_PASSWORD, SEED_CASHIER_PASSWORD, and SEED_PRODUCTION_PASSWORD. Existing accounts are not reset.
- Change admin password on first login.
- Auth login/refresh share a 10-request, 15-minute limit per IP; the API limit is 200 requests per minute per IP.
- Production CLIENT_URL defaults to `http://localhost:5000`; development uses `http://localhost:3000`.
- JWTs are verified with `jsonwebtoken.verify()`. Password hashes are never returned in ordinary API JSON.
- Product uploads accept JPEG/PNG/WebP MIME types, with the existing 5 MiB maximum. Binary/SQLite backups keep the existing 1 GiB maximum and SQLite file-signature checks.
- Security tests use a disposable database. The detailed secret inventory and route matrix are in `SECURITY-AUDIT-v1.6.6.md`.

3 — App Flow

Roles and default routes

Role	Default route
ADMIN	/dashboard
PRODUCTION_MANAGER	/production
CASHIER	/pos
STAFF	/unauthorized

Key flows

1. **Login:** Enter credentials → verify password → issue tokens → check role → redirect to the role's default page.
2. **POS:** Add items → complete sale → print receipt or generate token.
3. **Token:** Enter token number → print token slip → record the sale immediately through the existing POS flow.
4. **Order:** Create advance order → set items, delivery date, and advance → save → print slip. Existing permissions for order creation and status changes remain in place.
5. **Production:** Select recipe → enter quantity → view cost estimate → complete production → update stock and costs.
6. **HR:** Mark attendance, including backdated attendance → generate salary → populate deductions → print payslip.
7. **Supplier:** Receive stock → record payment with applicable advance deduction → print supplier receipt.
8. **Backup:** Select groups → choose folder → back up now or configure the automatic schedule.
9. **Restore:** Upload supported database backup → confirm → restore → reload/sign in again. SQLite [.db](#) and legacy PostgreSQL [.dump](#) formats follow the existing implementation's supported paths; the desktop restore validates SQLite content.

4 — UI/UX Design

Palette — existing design direction, unchanged

Purpose	Palette
Primary	Dark teal/green, including sidebar active state
Accent	Gold/amber for location text and highlights
Background	Warm cream/beige
Surface	White cards
Text	Dark brown
Error	Red
Success	Green

This is the supplied design reference. Existing component-specific colors and dark-mode styling remain untouched by the security release.

Typography

- System sans-serif body text, 14px base design reference.
- Monospace typography for receipt/slip printing.

Component rules

- Modals: blurred backdrop, centered, scrollable with a 90vh maximum height.
- Slips: 80mm width, monospace, bold amounts, logo at top, Viralage footer.
- Collapsed sidebar: centered icons in a 44×44px hover target.
- Number inputs: spinner arrows removed by global CSS.
- Printing: use the existing `silentPrint()` flow without a Windows print dialog.

These are the supplied UI conventions. This release does not redesign components or change printing behavior.

5 — Backend Schema

Core models

Domain	Models
Identity and configuration	User, ShopSettings, AuditLog
Catalog and inventory	Category, Product, RawMaterial, StockMovement
Purchasing and suppliers	Supplier, PurchaseOrder, PurchaseItem, PurchaseReturn, PurchaseReturnItem, SupplierPayment, SupplierAdvance, SupplierAdvanceRecovery
Customers and orders	Customer, Order, OrderItem
Sales	Sale, SaleItem, SaleReturn, SaleReturnItem, Token, TokenCounter
Expenses and accounting	Expense, ChartOfAccounts, JournalEntry, JournalLine
HR and payroll	Employee, Attendance, Salary, SalaryRevision, EmployeeAdvance, EmployeeLoan, LoanRecovery, EmployeeFine, LeaveRequest
Recipes and production	Recipe, RecipeIngredient, ProductionOrder, ProductionConsumption
Backups	BackupHistory, BackupSchedule

The authoritative schema is [prisma/schema.prisma](#). The server's Prisma folder is copied from it during the release build.

Relationships

- Products belong to categories; sale/order items reference products.
- Suppliers connect to purchases, payments, advances, and recoveries.
- Recipes contain raw-material ingredients; production orders record planned/actual consumption and finished output.
- Employees connect to attendance, salary, revisions, leave, advances, loans, recoveries, and fines.
- Journal entries contain journal lines linked to chart-of-accounts records.
- User relations identify cashiers, creators, approvers, and other authorized operators.

Global API rules

- Errors use { `success: false`, `message: string` }. Internal exceptions, query details, hashes, and stack traces are not returned.
- The requested list convention is pagination with default 20 and maximum 100. **Current compatibility exception:** existing selectors and some reports return complete lists or use other limits; this release preserves those response contracts.
- Every write passes numeric validation before database operations, with endpoint-specific validation retained. Authentication/user-management writes additionally use express-validator.
- Never return password fields in ordinary API responses. Authorized backup files necessarily retain database data for restoration.
- 401 means not authenticated; 403 means not authorized.
- Protected routers use `authenticate()` and sensitive routes add `authorize()`.
- Public entry points are `/api/auth/login`, `/api/auth/refresh-token` (which verifies the refresh token), and `/api/health` for desktop startup/monitoring. Logout now requires authentication.
- ADMIN alone can delete employees, reset business data, view audit logs, manage users, and write accounting data.
- CASHIER cannot access employee, salary, or accounting APIs. PRODUCTION_MANAGER cannot access sale or salary APIs.
- Single-shop entity IDs remain available for existing UI actions. Sensitive data access is restricted by server-side role checks.

6 — Implementation Plan: Completed Phases

Phase	Completed scope
0	Project setup, Prisma schema, Express server, local database; legacy Docker configuration
1	JWT authentication and role-based access for Admin, Production Manager, and Cashier
2	POS billing, inventory, raw materials, stock movements
3	Recipes/BOM, production orders, costing engine
4	HR: employees, attendance, leave, payroll, advances, loans
5	Chart of accounts, automatic journal entries, accounting reports
6	Advance orders, customer profiles, supplier management
7	Backup/restore, electron-updater integration, ADMIN-protected business-data reset
8	Security hardening, numeric/auth validation, rate limiting, headers, role checks, upload and error protections

Current version: 1.6.6, read from `desktop/package.json`.

Release verification

The server compiles successfully. The isolated security suite passes 98 assertions, including a valid cashier sale and inventory decrement. It checks authentication, role restrictions, bcrypt storage, token lifetimes/signatures, input rejection, upload MIME rejection, error masking, CSP/CORS, rate limits, and parameterized SQLite backup/merge.

Release outputs

- `desktop/release/Darbar Sweets Setup 1.6.6.exe`
- `desktop/release/win-unpacked/`
- `desktop/release/latest.yml`

Build command: `scripts\build-release.bat`.